

Bài Kiểm tra về tư duy lập trình

Bài kiểm tra này nhằm đánh giá năng lực của ứng viên trong việc sử dụng framework NestJS, lập trình thuật toán cơ bản và phát triển server game đơn giản. Ứng viên cần thể hiện khả năng viết mã sạch, tư duy logic và triển khai các tính năng theo yêu cầu. Bài kiểm tra gồm ba phần:

Bài 1: Nghiên cứu và Triển khai API với NestJS

Mục tiêu: Đánh giá khả năng sử dụng NestJS để xây dựng API RESTful cơ bản theo mô hình MVC.

Tài liệu tham khảo: Tài liệu NestJS

Yêu cầu:

1. Hiểu biết về NestJS:

- Giải thích ngắn gọn vai trò của các thành phần chính trong NestJS: Modules, Controllers, và Services.
- Mô tả cách NestJS sử dụng TypeScript để hỗ trợ phát triển ứng dụng.

2. Triển khai API:

- Xây dựng một API RESTful để quản lý thực thể "Task" với các trường: id (UUID), title (chuỗi), description (chuỗi), status (enum: "To Do", "In Progress", "Done"), và createdAt (thời gian).
- Triển khai các thao tác CRUD:
 - Create:** Tạo task mới.
 - Read:** Lấy danh sách task hoặc một task cụ thể theo ID.
 - Update:** Cập nhật thông tin task.
 - Delete:** Xóa task.
- Sử dụng TypeORM với cơ sở dữ liệu SQLite hoặc MongoDB với Mongoose để lưu trữ dữ liệu.
- Tổ chức ứng dụng theo mô hình MVC:
 - Model:** Định nghĩa schema Task.
 - Controller:** Xử lý các yêu cầu HTTP.
 - Service:** Chứa logic nghiệp vụ cho Task.

3. Yêu cầu bổ sung:

- Thêm xác thực đầu vào cơ bản (ví dụ: đảm bảo title không rỗng) bằng Pipes.
- Tài liệu hóa API bằng Swagger (OpenAPI) với endpoint /docs.
- Viết ít nhất một unit test (sử dụng Jest) cho Service hoặc Controller.

4. Tối ưu hóa:

- Đảm bảo API phản hồi nhanh (dưới 200ms) cho các yêu cầu GET với 100 bản ghi.

Sản phẩm cần nộp:

- Mã nguồn ứng dụng NestJS, tổ chức theo cấu trúc thư mục rõ ràng.
- Tài liệu ngắn (Markdown) giải thích cách chạy ứng dụng và truy cập Swagger.
- Kết quả chạy unit test.

Tiêu chí đánh giá:

- Tính chính xác và đầy đủ của các tính năng CRUD.
- Chất lượng mã nguồn (sạch, dễ đọc, tuân thủ Clean Code cơ bản).
- Tính hoàn chỉnh của tài liệu API và unit test.
- Khả năng sử dụng TypeScript và cấu trúc MVC.

Bài 2: Tính Số Fibonacci Thứ 50

Mục tiêu: Đánh giá khả năng viết thuật toán cơ bản và tối ưu hóa hiệu suất bằng JavaScript.

Yêu cầu:

1. Triển khai thuật toán:

- Viết hàm tính số Fibonacci thứ 50 ($F(50)$) bằng JavaScript, sử dụng **Dynamic Programming** với memoization hoặc mảng.
- Sử dụng BigInt để xử lý số lớn.
- Đo thời gian thực thi bằng console.time và console.timeEnd, đảm bảo thời gian thực thi trung bình dưới **1ms** trên CPU 2.5 GHz.

2. Kiểm tra tính đúng đắn:

- Kiểm tra kết quả với các giá trị $n = 10, 20$, và 50.
- Báo cáo thời gian thực thi trung bình qua 10 lần chạy.

3. Phân tích:

- Giải thích ngắn gọn độ phức tạp thời gian và không gian của thuật toán ($O(n)$ cho thời gian, $O(n)$ hoặc $O(1)$ cho không gian tùy phương pháp).

Sản phẩm cần nộp:

- Mã nguồn JavaScript cho thuật toán.
- Báo cáo ngắn (Markdown) mô tả thuật toán, độ phức tạp, và kết quả thời gian thực thi.
- Kết quả kiểm tra với các giá trị n .

Tiêu chí đánh giá:

- Tính chính xác của kết quả $F(50)$.
- Hiệu suất thuật toán (thời gian thực thi $< 1ms$).
- Chất lượng giải thích độ phức tạp.
- Tính rõ ràng và chú thích của mã nguồn.

Bài 3: Phát triển Server Game Đơn Giản với NestJS

Mục tiêu: Đánh giá khả năng xây dựng server game cơ bản với NestJS, tập trung vào quản lý tài khoản và hai trò chơi: Line 98 và Cờ Caro X O.

Yêu cầu chung:

- Sử dụng NestJS để triển khai server, tuân thủ mô hình MVC.
- Sử dụng SQLite (với TypeORM) hoặc MongoDB (với Mongoose) để lưu trữ dữ liệu.
- Sử dụng WebSocket (hoặc Socket.IO) cho các tính năng thời gian thực.
- Đảm bảo mã nguồn dễ đọc và có cấu trúc rõ ràng.

Chức năng quản lý tài khoản:

1. Đăng ký và đăng nhập:

- Cho phép người dùng đăng ký và đăng nhập bằng username và password.
- Mã hóa mật khẩu bằng bcrypt.
- Tạo JWT token khi đăng nhập để xác thực các yêu cầu API.

2. Cập nhật thông tin:

- Cho phép thay đổi email và nickname.
- Chỉ người dùng đã đăng nhập mới được cập nhật thông tin.

Trò chơi 1: Line 98

Yêu cầu chức năng:

- Logic trò chơi:
 - Lưới 9×9 với 5 màu bóng.
 - Người chơi di chuyển bóng để tạo hàng 5 bóng cùng màu (ngang, dọc, chéo) để xóa.
 - Sinh ngẫu nhiên 3 bóng mới sau mỗi lượt.
- Giao diện đồ họa:
 - Sử dụng HTML5 Canvas hoặc một thư viện frontend đơn giản để hiển thị bàn chơi.
 - Thêm hiệu ứng chọn bóng (phóng to) và di chuyển cơ bản.
- Tính năng trợ giúp:

- Cung cấp nút trợ giúp để đề xuất một nước đi hợp lệ ngẫu nhiên.

Yêu cầu kỹ thuật:

- Sử dụng WebSocket để đồng bộ trạng thái bàn chơi.
- Lưu trạng thái trò chơi vào cơ sở dữ liệu.

Trò chơi 2: Cờ Caro X O

Yêu cầu chức năng:

- **Logic trò chơi:**
 - Bàn chơi 15x15, hai người chơi đánh X hoặc O, thắng khi tạo hàng 5 ký hiệu liên tiếp.
 - Thông báo người thắng và kết thúc ván đấu.
- **Chế độ trực tuyến:**
 - Hỗ trợ hai người chơi đấu với nhau qua WebSocket.
 - Hỗ trợ ghép cặp ngẫu nhiên.
- **Giao diện đồ họa:**
 - Giao diện cơ bản bằng HTML5 Canvas hoặc thư viện frontend.
 - Hiển thị lượt chơi hiện tại và kết quả.

Yêu cầu kỹ thuật:

- Sử dụng WebSocket để đồng bộ lượt đi.
- Lưu lịch sử trận đấu vào cơ sở dữ liệu.

Yêu cầu bổ sung:

- Triển khai ít nhất một unit test cho mỗi trò chơi.
- Đảm bảo server xử lý được ít nhất 10 người chơi đồng thời mà không bị trễ (latency < 200ms).

Sản phẩm cần nộp:

- Mã nguồn server (NestJS) và client (giao diện cơ bản).
- Tài liệu ngắn (Markdown) hướng dẫn chạy ứng dụng.
- Kết quả chạy unit test.

Tiêu chí đánh giá:

- Tính hoàn chỉnh của các chức năng (quản lý tài khoản, Line 98, Cờ Caro).
- Chất lượng mã nguồn (tuân thủ MVC, dễ đọc).
- Độ ổn định và hiệu suất của server.
- Tính rõ ràng của tài liệu và unit test.

Lưu ý Chung

- Nộp mã nguồn qua kho Git (GitHub/GitLab) với README hướng dẫn chạy ứng dụng.
- Sử dụng **TypeScript** hoặc JavaScript, ưu tiên TypeScript.
- Ứng dụng phải chạy được trên môi trường local với SQLite hoặc MongoDB.

Chúc các ứng viên hoàn thành bài kiểm tra tốt!